How to Defend Serverside Programming Practical Works 1–6

Database design · relations and JOINs · indexes and EXPLAIN · ORM setup · entities, Criteria, repositories · REST and transactions

Серверное программирование · variant 1 · the «ВелоГрад» bike-rental database · English edition

This guide prepares you for the oral defense of all six practical works of the Serverside Programming course. For each work you get a 30-second pitch, a demo script, the things you must be able to explain, and likely questions with answers.

It matches the current, minimal code. On 5 November every exercise folder was reduced to exactly what its task sheet asks for: no comments, no tests, no extra tools, and one command per exercise (`npm start`). The six reports were rebuilt from that code and from fresh runs, so they now match the code line by line. Every command and number in this guide was run or measured on 5 November on a copy of the database; where something needs care, the guide says so.

I do not know exactly how your teacher runs a defense. The usual format is: the teacher opens your report, asks you to show a result, changes a query or a parameter a little and asks why it behaves as it does. Everything below is built for that. Each question is shown in English and, under it, in the Russian wording you are likely to hear; most answers have a short Russian version (“Say it in Russian”). All Russian text is in normal type, not italics.

Do not memorise word for word. Read an answer, close the guide, and say it in your own words; if you can, you understand it.

Contents

- 0. Before you go: what the tasks asked, where everything is, how to run it, checklist
- 1. Work No. 1 – database design (SQLite)
- 2. Work No. 2 – relations, JOINs, data generation (PostgreSQL)
- 3. Work No. 3 – indexes and EXPLAIN ANALYZE
- 4. Work No. 4 – ORM setup: DAO, sessions, transactions (Drizzle instead of Hibernate)
- 5. Work No. 5 – entities, Criteria API, repositories
- 6. Work No. 6 – REST service and transactions (Express instead of Spring Boot)
- 7. If something goes wrong
- 8. Cheat sheet: numbers and rules
- 9. Glossary English – Russian

0. Before you go

What the task sheets asked, and where it is

All paths are inside `C:\Users\USER\Documents\university-serverside-by-exercise`. Folder names: **pz1** = `pz1-database-design`, **pz2** = `pz2-relations-joins-generation`, **pz3** = `pz3-indexes-explain`, **pz4** = `pz4-orm-setup`, **pz5** = `pz5-entities-criteria-repositories`, **pz6** = `pz6-rest-transactions`. Each folder also has `report\` (the PDF report) and `task\` (the text of the task sheet).

Work	Requirement	Where it is	Status
1	ER or UML diagram, at least 5 tables	pz1\er-diagram.svg – 7 tables	Done
1	Schema with types and primary keys; at least 10 rows per table	pz1\sql\01_schema.sql, 02_seed.sql (10, 12, 14, 12, 10, 16, 12 rows)	Done
1	CROSS JOIN over all the tables, with the result	pz1\sql\03_cross_join.sql – 38,707,200 rows	Done
2	Relation type for each pair; FK, UNIQUE, junction table	pz2\sql\01_schema.sql, 03_relations.sql; table in the report	Done
2	INNER, LEFT, FULL OUTER, CROSS JOIN with explanation	pz2\sql\04_joins.sql – 12, 13, 17 and 6 rows	Done
2	At least 10,000 generated records; typical query	pz2\sql\05_generate.sql + generate.sql (28,139 records, 10,016 rentals); 06_typical_query.sql	Done
3	At least 500,000 records	pz3\sql\01_load.sql – 967,801 records	Done
3	Query: 3+ tables, 3+ WHERE conditions of different types	pz3\sql\02_query.sql – 3 tables, 4 conditions (text, text, number, date)	Done
3	Plan and time before; indexes, REINDEX, measure again; table	pz3\sql\03_indexes.sql + 04_explain.sql; table of measurements in the report	Done
4	Config, DAO (getSession, begin, commit), TestSrvs with 3 methods, Main	pz4\src\config.ts, DAO.ts, TestSrvs.ts, Main.ts, schema.ts	Done (Drizzle instead of Hibernate)
5	Entity for all tables; Criteria findAll; dynamic filter; CRUD; 1:N demo	pz5\src\model.ts (9 tables), relations.ts, Criteria.ts, StationDao.ts, DistrictDao.ts	Done
5	Repository; query method; @Query	pz5\src\JpaRepository.ts, StationRepository.ts, DistrictRepository.ts	Done (hand-written, see 5.4)
6	GET, POST, DELETE; JSON; DTO from several tables	pz6\src\app.ts, services.ts (RentalService → RentalDto)	Done (Express instead of Spring Boot)
6	Transactional service; rollback demo; isolation level	pz6\src\DAO.ts (transactional), services.ts (register), demo.ts; READ COMMITTED	Done
6	Demo client	pz6\public\index.html	Done

About the stack: section C of every task sheet allows an alternative stack agreed with the teacher; you told me the teacher accepted Node.js and TypeScript. Work 1 is on SQLite; works 2 and 3 on PostgreSQL 17 (the reference DBMS); works 4–6 use Drizzle ORM instead of Hibernate and Express instead of Spring Boot, with the Java-to-TypeScript mapping written in each report. Nothing was uploaded to eКурсы by me; you upload the files yourself.

Old and new reports. If you already uploaded the earlier reports (the ones in `Downloads` without a folder, made before 5 November), they describe the earlier, fuller version of the code (tests, extra scripts, validation). The rebuilt reports in the exercise folders describe the current code. Make sure you defend the code that your report describes, and open the matching files.

How to run it

Works 2–6 use a separate PostgreSQL server whose data lives in the folder `.pgdata` (about 1 GB; port 54329, local only, no password); your computer's main PostgreSQL is not touched. Run in PowerShell:

```
cd C:\Users\USER\Documents\university-serverside-by-exercise\pz3-indexes-explain
npm run pg:start        # start the server (works from the folder of any of works 2-6)
$env:PGCLIENTENCODING = 'UTF8'    # so Russian letters in psql are not shown as ????
& 'C:\Program Files\PostgreSQL\17\bin\psql.exe' -h 127.0.0.1 -p 54329 -U postgres -d velograd
npm run pg:stop         # when you are done
```

Work	Command (inside its folder)	What it does
1	`npm start`	creates `velograd.db` (SQLite), runs the schema and the data, prints row counts, runs the three CROSS JOIN queries. No server needed.
2	`npm run pg:start`, `npm start`	runs the six SQL files in order. **Rebuilds the whole database from scratch** (drops all tables); see the warning below.
3	`npm start`	loads data if there are fewer than 100,000 rentals, then runs the query and the six index steps with an EXPLAIN ANALYZE after each (about 10 seconds on the full database).
4	`npm start`	prints a station name, a list of names, and adds one station (each run leaves one more station in the table).
5	`npm start`	runs every DAO and repository method and prints the results (it removes what it creates).
6	`npm start`; `npm run demo`	start: the REST server on port 8080 (open `http://localhost:8080/`); demo: runs the whole scenario in one process, including the rollback, and cleans up.

- **The first start can take a while.** If the computer was switched off or crashed since the last time, PostgreSQL recovers before it accepts connections; this took about a minute once. Start the server well before the defense.
- **Do not run `npm start` in pz2 just before the defense.** It drops and recreates the tables with the small data set (about 28,000 records). To get the 400,000 rentals back, run `npm start` in pz3 afterwards (it detects the small data set, loads it again, about a minute). Use psql for live demos instead (below).
- Inside psql, `\i path` runs a file from the folder you started psql in, for example `\i sql/04_explain.sql` (start psql from the pz3 folder); `\q` quits.
- **Work 3 depends on work 2:** pz3 adds rows to the tables that pz2 created, so pz3 cannot start on an empty server; the shared `.pgdata` already has everything.
- **Do not run the whole file `sql/04_joins.sql`** of pz2 on the full database: its FULL OUTER JOIN would print 400,000 rows. It was written for the 12 original stations. Use the small queries in section 2.2 instead.
- Only one server can use port 54329. If `pg:start` says the port is busy, the original project's server may be running: stop it first.

15-minute checklist

- Start the server and connect with psql. Run `\dt`: 9 tables. Run `SELECT count(*) FROM rental;`: 400000. Run `\di idx_*`: five indexes (if fewer, run `npm start` in pz3, it recreates them).
- From the pz3 folder run `\i sql/04_explain.sql` **twice**: the first run in a new psql session is slower (cold caches, about 4 ms); the second shows about 1 ms.
- Open the six reports once. Know the page that shows the ER diagram (report 1), the JOIN results (report 2), the measurement table (report 3), the DAO (report 4), the repositories (report 5) and the rollback (report 6).

- Read each 30-second pitch aloud once. Remember the key numbers (section 8).
- Run `npm start` in pz1 once (no server needed) and `npm run demo` in pz6 once to see that everything works.

How to behave during the defense

- Start with the 30-second pitch, then demo. That way you steer the conversation.
- If you do not know an answer, say what you do know and name the limit honestly: “I did not measure that, but I would expect…”.
- If the teacher asks you to change a query, change it calmly in psql and run it again; for plans put `EXPLAIN (ANALYZE, BUFFERS)` in front.
- If your times differ from the report, that is normal: absolute times depend on the computer and on whether it is a fresh psql session; the ratio and the type of scan are what matter.
- The code is deliberately minimal: if asked “why no validation / tests / comments?”, answer that the task sheets ask for exactly this and the code answers only the questions asked.

1. Work No. 1 – database design

The task: choose a domain, draw an ER/UML diagram (at least 5 tables), create the schema with types and primary keys, fill every table with at least 10 rows, and write a CROSS JOIN over all the tables.

1.1 The 30-second pitch

Say this

"The domain is a city bike-rental network, 'ВелоГрад'. The model has seven tables: district, station, bicycle, client, tariff, rental and payment. The relations are foreign keys; integrity is guaranteed by primary keys, NOT NULL, UNIQUE and CHECK constraints, for example the phone and e-mail format, allowed statuses and non-negative amounts. I used an agreed alternative stack: SQLite and TypeScript on Node.js. The folder has the schema script, the data script and a CROSS JOIN script that first computes the expected number of rows, then counts the real result, and prints five rows. The result is 38,707,200 rows, exactly the product of the table sizes."

Предметная область – городская сеть велопроката «ВелоГрад». В модели семь таблиц: район, станция, велосипед, клиент, тариф, поездка и платёж. Связи – внешние ключи; целостность обеспечивают первичные ключи, NOT NULL, UNIQUE и CHECK. Задание выполнено на согласованном стеке: SQLite и TypeScript. CROSS JOIN даёт 38 707 200 строк – ровно произведение числа строк таблиц.

1.2 The seven tables

Table	Rows	What it holds	Links
district	10	city districts	one district has many stations
station	12	rental stations (bike parks) with address, capacity, coordinates	→ district
bicycle	14	the fleet: serial number, model, kind, status, odometer	→ station (home station)
client	12	registered users: name, phone, e-mail, birth date	–
tariff	10	tariff plans: unlock price, price per minute, free minutes	–
rental	16	rides; unfinished rides have no end station and no cost	→ client, bicycle, tariff, start station, end station
payment	12	payments for finished paid rides: method, status, amount	→ rental (one to one)

1.3 Demo script

1. In the pz1 folder run `npm start`. It prints the row counts of the seven tables, then the three queries with their results.
2. Show the first result: the product of the table sizes, 10·12·14·12·10·16·12 = 38,707,200 (the query computes it from the real counts). Show the second: the real `COUNT(*)` of the CROSS JOIN, 38,707,200 – the two match.
3. Show the third: only five rows are printed (`LIMIT 5`), because the full output would be 38.7 million rows of 55 columns.
4. Open `er-diagram.svg` (it was generated from the real schema) and point at the links.
5. Open `sql\01_schema.sql` and show the constraints; open report 1 (8 pages).

1.4 What you must be able to explain

- The seven tables and how they are linked (table above).
- Which constraints protect the data: primary keys, foreign keys (ON DELETE RESTRICT; a payment is removed together with its rental), NOT NULL, UNIQUE, CHECK.
- Why a CROSS JOIN returns the product of the table sizes, and what that product is: 10·12·14·12·10·16·12 = 38,707,200 rows with 55 columns.
- Why SQLite, and that works 2–3 moved the same schema to PostgreSQL.

1.5 Questions and answers

What are primary and foreign keys?
Что такое первичный и внешний ключ?

A primary key identifies a row uniquely: it is unique and never empty. A foreign key is a column whose values must match the primary (or a unique) key of another table, so the database refuses a reference to a row that does not exist, for example a rental of a client who does not exist.

Say it in Russian: «Первичный ключ однозначно идентифицирует строку. Внешний ключ обязан совпадать со значением ключа другой таблицы, поэтому нельзя сослаться на несуществующую запись.»

Which tables do you have and how are they related?
Какие таблицы в вашей модели и как они связаны?

A district contains stations; bicycles have a home station; a rental ties a client, a bicycle and a tariff to a start station and an end station; a payment belongs to exactly one rental.

Say it in Russian: «Район содержит станции, на станциях стоят велосипеды. Поездка связывает клиента, велосипед и тариф, у неё есть станция начала и станция конца. Платёж относится к одной поездке.»

What does CROSS JOIN do and how many rows does it return?
Что делает CROSS JOIN и сколько строк он вернёт?

It builds the Cartesian product: every row of the left table is paired with every row of the right table, with no join condition. The row count is the product of the table sizes: for all seven tables 38,707,200 rows of 55 columns. The script computes that number arithmetically first, then counts the real result (they match), and prints only five rows. An accidental CROSS JOIN of big tables is a classic mistake because the result grows so fast.

Say it in Russian: «Декартово произведение: каждая строка левой таблицы сочетается с каждой строкой правой. Число строк равно произведению числа строк таблиц.»

Why SQLite and not PostgreSQL?
Почему SQLite, а не PostgreSQL?

The task sheet allows an alternative stack (section C), and your teacher agreed to TypeScript on Node.js. SQLite needs no server, so work 1 runs with one command. Works 2 and 3 use PostgreSQL, the reference DBMS of the course, with the same schema ported over; report 2 lists the differences between the two versions.

Say it in Russian: «Методичка допускает альтернативный стек. SQLite не требует сервера, поэтому первая работа запускается одной командой. Следующие работы выполнены на PostgreSQL.»

Why is payment a separate table?
Зачем платёж вынесен в отдельную таблицу?

A payment is a fact of its own with its own attributes: method, status, time, external id. A rental may have no payment yet (unfinished or free), and storing empty payment columns in the rental table would be wasteful. The relation is one to one, so `rental_id` in the payment table is UNIQUE.

Say it in Russian: «Платёж – самостоятельный факт со своими атрибутами, и его может не быть. Связь один к одному, поэтому rental_id уникален.»

Which integrity rules did you add besides the keys?
Какие ограничения целостности вы добавили, кроме ключей?

UNIQUE where the data demands it (serial numbers, phones, e-mails, tariff codes). CHECK rules: station capacity from 4 to 60, valid coordinates, a phone that starts with +7 and has 12 characters, the e-mail shape, allowed statuses, non-negative prices and distances, `finished_at` later than `started_at`. One combined rule: an unfinished rental has no end station and no cost, a finished one has both. SQLite does not index foreign keys by itself, so the script also creates those indexes.

Say it in Russian: «Кроме ключей добавлены NOT NULL, UNIQUE и CHECK: формат телефона, допустимые статусы, неотрицательные суммы. У незавершённой поездки нет станции конца и стоимости.»

Why do dates look like text in the SQLite schema?
Почему в схеме SQLite даты хранятся как текст?

SQLite has no separate date type: dates and times are stored as TEXT in ISO format, and a CHECK with `date()` or `datetime()` rejects anything that is not a valid date. In PostgreSQL (work 2) the same columns are real `date` and `timestamp` types.

2. Work No. 2 – relations, JOINs, data generation

The task: decide the relation type for each pair of tables; implement the relations (foreign keys, UNIQUE, junction tables for many-to-many); show INNER, LEFT, FULL OUTER and CROSS JOIN; generate at least 10,000 records; run a typical application query.

2.1 The 30-second pitch

Say this

"I moved the schema to PostgreSQL 17 and implemented every kind of relation: one-to-many with foreign keys; one-to-one with a UNIQUE key on the payment and with a primary key that is also a foreign key in the client's wallet; many-to-many with a junction table of favourite stations with a composite primary key; and a recursive relation, 'a client invited a client'. The ON DELETE action follows the meaning: RESTRICT for reference data and history, CASCADE for dependent data, SET NULL for a soft link. Then I showed INNER, LEFT, FULL OUTER and CROSS JOIN on the base data and generated more than 28 thousand records, 10,016 of them rentals, with the database's own tools: generate_series and random() with a fixed seed, so the data is reproducible. The typical application query is a client's ride history; it uses four tables."

Я перенёс схему в PostgreSQL 17 и реализовал связи всех типов: один ко многим, один к одному, многие ко многим через промежуточную таблицу и рекурсивную. Действие при удалении выбрано по смыслу: RESTRICT, CASCADE, SET NULL. Показаны INNER, LEFT, FULL OUTER и CROSS JOIN. Сгенерировано более 28 тысяч записей через generate_series и random() с фиксированным зерном.

2.2 Demo script

`npm start` in pz2 shows everything, but it rebuilds the database. For a live demo on the full database use psql and these queries (verified on the full database; they restrict the rows so that the numbers match the report):

```
-- INNER: only matching pairs (district 9 has no stations, so it is missing)          -> 12 rows
SELECT count(*) FROM district d INNER JOIN station s ON s.district_id = d.id WHERE s.id <= 12;

-- LEFT: all districts, NULL on the station side for district 9                      -> 13 rows
SELECT count(*) FROM district d LEFT JOIN station s ON s.district_id = d.id AND s.id <= 12;

-- FULL OUTER: unmatched rows of both sides                                           -> 17 rows
--   14 matched pairs + 1 station nobody rode to + 2 rentals that are not finished
SELECT count(*) FROM (SELECT * FROM station WHERE id <= 12) s
FULL OUTER JOIN (SELECT * FROM rental WHERE id <= 16) r ON r.end_station_id = s.id;

-- CROSS: every district with every tariff                                            -> 100 rows
SELECT count(*) FROM district CROSS JOIN tariff;
```

To *show rows* instead of counts, replace `count(*)` with the columns you want, for example `d.name, s.name`. The file `sql/04_joins.sql` has the same four JOINs (CROSS JOIN limited to two districts and three tariffs, 6 rows); its output is in report 2.

Then the integrity experiments, each in a transaction that is rolled back (all verified; the errors are expected):

```
-- RESTRICT: a district with stations cannot be deleted
BEGIN; DELETE FROM district WHERE id = 1; ROLLBACK;
-- ERROR: update or delete on table "district" violates foreign key constraint "fk_station_district"
on table "station"

-- UNIQUE (one to one): a second payment for the same rental is refused
BEGIN;
INSERT INTO payment (rental_id, amount, method, status, paid_at, external_id)
SELECT rental_id, amount, 'card', 'succeeded', now(), 'dup-test' FROM payment WHERE id = 1;
ROLLBACK;
-- ERROR: duplicate key value violates unique constraint "payment_rental_id_key"

-- CASCADE: deleting a rental removes its payment (1 -> 0), the rollback brings it back (1)
BEGIN;
SELECT count(*) FROM payment WHERE rental_id = 2;  DELETE FROM rental WHERE id = 2;  SELECT count(*)
FROM payment WHERE rental_id = 2;
ROLLBACK;
```

1. Run `\dt` (9 tables) and `\d rental` (the foreign keys at the bottom).
2. Run the four JOIN queries above; say what each count means.
3. Run the RESTRICT, UNIQUE and CASCADE experiments.
4. Open `sql\generate.sql`: point at `setseed`, `generate_series`, `random()` and the parent-before-child order; then `sql\06_typical_query.sql` and run it with `\i sql/06_typical_query.sql` (it picks the client with the most rides: client 963, 8 rows are printed).

2.3 The relations in your model

Type	Example	How it is implemented	ON DELETE
one to many	district → station, station → bicycle, client / bicycle / tariff → rental, station → rental (twice: start and end)	foreign key on the “many” side	RESTRICT
one to one	rental → payment	foreign key + UNIQUE (`payment.rental_id`)	CASCADE: the payment goes with its rental
one to one	client → client_wallet	the primary key `client_id` is also the foreign key	CASCADE
many to many	client ↔ station (favourite stations)	junction table `client_favorite_station`, two foreign keys, composite primary key	CASCADE
recursive one to many	client → client (“who invited whom”)	`client.referred_by` refers to the same table	SET NULL
association entity	client ↔ bicycle	the table `rental` with its own attributes (time, cost)	RESTRICT

The schema has 9 primary keys, 12 foreign keys, 8 UNIQUE and 23 CHECK constraints (counted from the PostgreSQL system catalog).

2.4 What you must be able to explain

- The relation types and their implementation (table above).
- The ON DELETE actions: RESTRICT or NO ACTION forbids the delete; CASCADE deletes the children; SET NULL clears the reference; SET DEFAULT puts the default value.
- The difference between INNER, LEFT, RIGHT, FULL OUTER and CROSS JOIN, and the data on which it is visible: a district without stations, a station nobody rode to, two unfinished rentals.

- How the data is generated: `generate_series` produces rows, `random()` and lookup arrays produce values, `setseed` makes the result repeatable; parents first, then children.
- The generation plan: 60 stations, 600 bicycles, 2,000 clients with 2,000 wallets and 4,770 favourite pairs, 10,000 rentals, 8,591 payments.

2.5 Questions and answers

Which relation types exist and how do you implement each?
Какие виды связей есть и как реализовать каждый?

One to many – a foreign key in the "many" table (`rental.client_id` → client). One to one – a foreign key with UNIQUE (`payment.rental_id`) or a primary key that is also a foreign key (`client_wallet.client_id`). Many to many – a junction table with two foreign keys and a composite primary key (`client_favorite_station`). Recursive – a foreign key to the same table (`client.referred_by`).

Say it in Russian: «Один ко многим – внешний ключ в таблице «многие». Один к одному – внешний ключ с UNIQUE. Многие ко многим – промежуточная таблица с двумя внешними ключами. Рекурсивная – внешний ключ на ту же таблицу.»

What do RESTRICT, CASCADE and SET NULL mean, and why did you pick them?
Что означают RESTRICT, CASCADE и SET NULL? Почему выбраны именно они?

RESTRICT forbids deleting a parent that still has children: reference data and history are protected, so you cannot delete a district with stations or a client with rentals. CASCADE deletes the dependent rows together with the parent: the payment of a deleted rental, the wallet and favourites of a deleted client have no meaning on their own. SET NULL clears the reference: if a client who invited someone is deleted, the invited client stays and only the link disappears.

Say it in Russian: «RESTRICT запрещает удалить родителя с дочерними строками. CASCADE удаляет зависимые строки вместе с родителем. SET NULL обнуляет ссылку.»

What happens if you delete a district that has stations?
Что произойдёт, если удалить район, в котором есть станции?

The database returns a foreign key violation, because the stations use RESTRICT (see the experiment in 2.2). Run inside a transaction that is rolled back, nothing changes.

How is one-to-one implemented, and why does the wallet need no UNIQUE?
Как реализован «один к одному» и почему в кошельке не нужен UNIQUE?

For the payment: a foreign key `rental_id` with UNIQUE, so a rental has at most one payment. For the wallet: `client_id` is the primary key and at the same time a foreign key to the client; a primary key is already unique, so a separate UNIQUE would be redundant.

How is many-to-many implemented?
Как реализована связь «многие ко многим»?

By the table `client_favorite_station`: two foreign keys (to the client and to the station) form the composite primary key. It forbids repeating the same client–station pair. One client can mark many stations, and many clients can mark the same station.

How do INNER, LEFT and FULL OUTER JOIN differ? Show it on your data.
Чем отличаются INNER, LEFT и FULL OUTER JOIN? Покажите на ваших данных.

INNER keeps only the pairs for which the condition holds: a district without stations is missing (12 rows). LEFT keeps every row of the left table and fills the missing right side with NULL: district ‘Красногорский’ appears (13 rows). FULL OUTER keeps unmatched rows from both sides: joining stations to rentals by the end station shows station ‘Заречье’ where nobody arrived and two unfinished rentals without an end station; 17 rows in total, 14 of them matched pairs.

Say it in Russian: «INNER оставляет только совпавшие пары. LEFT сохраняет все строки левой таблицы, недостающее заполняет NULL. FULL OUTER сохраняет несовпавшие строки обеих сторон.»

Where is CROSS JOIN used in practice?
Зачем нужен CROSS JOIN на практике?

To enumerate all combinations, for example tariff × district, or calendar × station for reports that must show zeros; also to multiply rows in data generators. The row count is the product of the sizes, so on big tables it is used with care.

How did you generate the data?
Как вы сгенерировали данные?

With an SQL script: `generate_series` gives the number of rows, `random()` and lookup arrays give the values, and `setseed(0.2026)` fixes the seed, so a second run produces the same set. Parents are created first (stations, bicycles, clients), then rentals that refer to existing rows, then payments for finished paid rentals. Everything runs in one transaction, followed by ANALYZE. The parameters are in `05_generate.sql`, the algorithm in `generate.sql`.

Say it in Russian: «Скриптом на SQL: generate_series даёт строки, random() и справочники дают значения, setseed делает результат повторяемым. Сначала родители, затем потомки.»

What are LATERAL and MATERIALIZED doing in the generator?
Что такое LATERAL и MATERIALIZED в генераторе?

LATERAL lets a subquery on the right side refer to the columns of the current left row: for each rental it picks one of the tariffs that was valid on that date. MATERIALIZED forces a CTE to be computed once and stored, so the random values are not re-rolled each time the CTE is read.

What is the typical application query and why those JOINs?
Какой типовой запрос приложения и почему такие соединения?

A client’s ride history: rental joined to the start station (INNER), the end station (LEFT, because an unfinished ride has none), the tariff (INNER) and the payment (LEFT, because free rides have none). It picks the client with the most rides (963, ‘Попова Милана’) and prints the eight latest rides.

What is the difference between IDENTITY and SERIAL?
Чем IDENTITY отличается от SERIAL?

IDENTITY is standard SQL (`GENERATED … AS IDENTITY`): the counter belongs to the column. SERIAL is an older PostgreSQL shortcut that creates a separate sequence. The BY DEFAULT variant accepts explicit ids, as when the base data was inserted with fixed ids; afterwards `setval` moves the counter to the maximum id so that new ids do not collide.

3. Work No. 3 – indexes and EXPLAIN ANALYZE

The task: load at least 500,000 records; write a typical heavy query (3+ tables, 3+ WHERE conditions of different types); record the plan and the time before optimisation; add indexes, rebuild them and measure again; build the table indexes → time → cost; draw conclusions.

3.1 The 30-second pitch

Say this

"I brought the database to 967,801 records: 400,000 rentals and 50,000 clients. I took a typical support query – three tables, four conditions of three types, sorting and LIMIT – and measured it with EXPLAIN ANALYZE without indexes: about 102 ms and a full read of the rental table. Then I added indexes one at a time. The planner ignored the index on the date and the ordinary index on the surname: the first condition matches 68.6 % of the rows, and the second cannot serve LIKE under a language collation. An index with text_pattern_ops sped up the client lookup but not the total time. The decisive one was the foreign-key index rental(client_id): the time fell to about 4.3 ms, that is 23.4 times faster, the plan cost from 9,692 to 1,750, and the pages read from 6,655 to 1,025. The index on start_station_id is useless for this query. REINDEX and ANALYZE changed nothing because the indexes were fresh. The conclusion: index by the query plan, not just in case, and measure every index."

Я довёл базу до 967 801 записи. Типовой запрос – три таблицы, четыре условия, сортировка и LIMIT – без индексов выполнялся около 100 мс. Решающим оказался индекс внешнего ключа rental(client_id): время упало примерно в 23 раза. Остальные индексы для этого запроса не помогли. Вывод: индексы создают по плану запроса и каждый проверяют замером.

3.2 The query

```
SELECT c.last_name, c.first_name, s.name AS start_station, r.started_at, r.distance_km, r.total_cost
FROM rental  AS r
JOIN client  AS c ON c.id = r.client_id
JOIN station AS s ON s.id = r.start_station_id
WHERE c.last_name LIKE 'Кузн%'              -- text: prefix search
  AND c.first_name = 'Мария'                -- text: equality
  AND r.total_cost BETWEEN 100 AND 600      -- number
  AND r.started_at >= timestamp '2025-01-01' -- date and time
ORDER BY r.started_at DESC
LIMIT 100;
```

Scenario: support staff look up a client by surname and first name and review her paid rides, newest first. The query returns 71 rows. 962 clients have a surname starting with 'Кузн', and 31 of them are called 'Мария'. 176,557 of 400,000 rentals cost between 100 and 600; 274,493 (68.6 %) started in 2025 or later. (The SQL file has no comments; the comments above are only for you.)

3.3 Demo script (in psql, started from the pz3 folder)

1. `\i sql/04_explain.sql` – the plan with all indexes: Nested Loop, Index Scan using `idx_rental_client_id`. Run it twice or three times: the first run in a new session shows about 4 ms (cold caches), the next ones about 1 ms.
2. `DROP INDEX idx_rental_client_id;` and run `\i sql/04_explain.sql` again: the plan changes to Parallel Seq Scan on rental with a Hash Join, and the time is about 90 ms (measured: 90–92 ms).
3. `\timing on`, `CREATE INDEX idx_rental_client_id ON rental (client_id);` (about a quarter of a second), then `ANALYZE rental;` and run 04_explain.sql again: about 1 ms. This shows cause and effect live; I ran exactly this sequence on 5 November and left the index in place.

4. Open report 3: the measurement table and the chart of time per step; explain each step. To replay the six steps, run `npm start` in pz3 (it drops the five indexes first, then recreates them one by one).
5. If asked about size: `SELECT pg_size_pretty(pg_relation_size('idx_rental_client_id'));` gives 3904 kB.

3.4 The measurements, step by step

Each time is the median of 5 runs after one warm-up run; every measurement used a **new psql session**, so it includes the start-up cost of a fresh session (about 3 ms). That is why the index variant shows 4.3 ms in the table but about 1 ms when you repeat the query in one session. Cost is the planner's estimate at the root of the plan; pages are buffers read (shared hit + read).

Step	Index added	Time, ms	Planning, ms	Root cost	Pages	What the plan does
0	none (only PK and UNIQUE)	101.7	7.4	9,691.5	6,655	Parallel Seq Scan on rental, Hash Join
1	rental(started_at)	98.6	8.1	9,691.5	6,655	index not used: the condition matches 68.6 % of the rows
2	client(last_name)	95.0	8.7	9,691.5	6,655	index not used for LIKE under the language collation
3	client(last_name text_pattern_ops)	94.6	9.7	9,010.4	6,051	client lookup uses the index (Bitmap Index Scan); the rental scan still dominates
4	rental(client_id)	4.3	10.5	1,750.3	1,025	Nested Loop with Index Scan on rental: the decisive step
5	rental(start_station_id)	4.3	10.8	1,750.3	1,025	not needed for this query
6	REINDEX + ANALYZE	4.3	10.8	1,750.1	1,025	no real change: indexes and statistics were fresh

The key ratios: time 101.7 / 4.3 = 23.4 times faster; cost 9,692 / 1,750 = 5.5 times lower; pages 6,655 / 1,025 = 6.5 times fewer. Index sizes: started_at 8,792 kB, last_name 376 kB (both variants), client_id 3,904 kB, start_station_id 2,840 kB. A single first run right after the indexes were dropped is slower still (about 162 ms): parallel workers start and caches warm up; the table uses the steady value.

3.5 How to read an EXPLAIN ANALYZE plan

- **cost=a..b** – the planner's estimate in abstract units: a is the cost before the first row, b the total. It is not milliseconds.
- **actual time=a..b**, **rows**, **loops** – what really happened: milliseconds to the first and last row, the number of rows, how many times the node ran (time and rows are per run).
- **Buffers: shared hit / read** – pages (8 KB) taken from the cache or read from disk.
- **Seq Scan** reads the whole table. **Index Scan** walks the index and fetches rows from the table. **Bitmap Index Scan + Bitmap Heap Scan** builds a map of matching pages and reads them in order. **Parallel** means several worker processes share the work.
- **Hash Join** builds a hash table from the smaller set and probes it with the other; **Nested Loop** looks up matches in the inner set for each row of the outer set, which is fast when the inner side has an index.
- **Planning Time** is the time to build the plan, **Execution Time** the time to run it. Read a plan bottom-up and from the inside out.

3.6 Questions and answers

What is an index and why use one?
Что такое индекс и зачем он нужен?

An additional structure (a B-tree by default in PostgreSQL) that keeps the values of a column sorted together with pointers to the rows. A lookup costs about log n steps, while without an index the whole table must be read. The price is disk space and slower inserts, updates and deletes.

Say it in Russian: «Индекс – дополнительная структура (по умолчанию B-дерево), где значения столбца хранятся отсортированными со ссылками на строки. Поиск быстрее полного чтения таблицы, но индекс занимает место и замедляет запись.»

What does your query do and which conditions does it have?
Как работает ваш запрос и какие условия в нём?

It joins rentals, clients and stations. The conditions: the surname starts with ‘Кузн’ (text, LIKE), the first name equals ‘Мария’ (text, equality), the cost is between 100 and 600 (number), the start is not earlier than 1 January 2025 (date and time). Then ORDER BY date descending and LIMIT 100. It returns 71 rows.

Why does the query take about 100 ms without indexes? What does the database do?
Почему без индексов запрос выполняется около 100 мс?

To find the rentals of the 31 matching clients, the database has to read all 400,000 rows of `rental`: there is no index on `client_id`, and PostgreSQL does not create indexes for foreign keys by itself. This is a Parallel Seq Scan by three processes over 6,655 pages. The work grows in proportion to the table size.

Say it in Russian: «Индекса по client_id нет, а PostgreSQL не создаёт индексы внешних ключей сам, поэтому приходится читать все 400 000 строк таблицы поездок.»

Why was the started_at index not used?
Почему индекс по started_at не применился?

The condition “not earlier than 1 January 2025” matches 274,493 of the 400,000 rows, which is 68.6 %. When most of the table has to be read, a sequential scan is cheaper than hopping through an index, so the planner is right to ignore it. The index would help for narrow ranges, such as one day.

Why did the ordinary B-tree on last_name not help with LIKE 'Кузн%'?
Почему обычный индекс по last_name не помог при LIKE?

The database was created with a language collation (ICU, ru-RU). Under such an order, strings that share a prefix are not guaranteed to be adjacent, so an ordinary B-tree cannot serve a prefix search. You need `text_pattern_ops`, which compares strings byte by byte. With the ‘C’ collation an ordinary index would have worked.

Say it in Russian: «База создана с языковым порядком сортировки, поэтому для префиксного поиска нужен индекс с классом операторов text_pattern_ops.»

Why did the text_pattern_ops index not speed up the whole query?
Почему индекс text_pattern_ops не ускорил запрос целиком?

It did speed up the client lookup (the plan now uses a Bitmap Index Scan on it) and the cost fell from 9,692 to 9,010. But the main work was the Parallel Seq Scan over 400,000 rentals, and while it remains the total stays at about 95 ms. You optimise the bottleneck of the plan first.

Why was rental(client_id) decisive?
Почему индекс по rental(client_id) оказался решающим?

Now the 31 clients are found through the client index, and for each of them the rentals are taken through the `client_id` index (about two per client) instead of reading the whole table. The plan became a Nested Loop with Index Scan: 4.3 ms instead of 101.7, cost 1,750 instead of 9,692, pages 1,025 instead of 6,655. The other conditions are checked on a few dozen rows only.

Say it in Russian: «Теперь по индексу находятся 31 клиент, и для каждого по индексу client_id берутся его поездки, а не читается вся таблица. Время около 4 мс против 100.»

Why is the start_station_id index unused?
Почему индекс по start_station_id не использован?

The station is joined from the rental side: for each of the 71 rows one lookup by the station's primary key, which is indexed automatically. An index on `rental(start_station_id)` would serve the opposite direction, "find all rentals from station X". For this query it is useless, it takes 2,840 kB and it slows inserts down.

What do REINDEX and ANALYZE do, and why did they not change the time?
Что делают REINDEX и ANALYZE и почему они не изменили время?

REINDEX rebuilds the indexes: after many changes and deletes they become bloated with empty space, and rebuilding makes them compact again. ANALYZE refreshes the statistics about value distribution, which the planner uses to estimate row counts and choose a plan; after a bulk load it is essential. Here the indexes were just created and the statistics were fresh, so the time stayed at 4.3 ms.

Say it in Russian: «REINDEX перестраивает индексы. ANALYZE обновляет статистику, по которой планировщик выбирает план.»

How does cost differ from actual time?
Чем cost отличается от actual time?

Cost is a forecast in abstract units (reading a page, processing a row, calling an operator) that the planner uses to pick the cheapest plan. Actual time is measured in milliseconds. They correlate but are not proportional: in step 4 the cost fell 5.5 times while the time fell 23.4 times.

The planner estimated about 24 rows but found 31. Why?
Почему оценка около 24 строк, а нашлось 31?

Statistics are built from a sample and treat the conditions as independent, so they know nothing about how first names and surnames go together. The difference is small and did not change the plan. If estimates are far off, ANALYZE, a larger statistics target or extended statistics on several columns help.

What do indexes cost, and which would you keep?
Какие недостатки у индексов? Что бы вы оставили?

They take space (the five indexes are about 16 MB next to 32 MB of rental data), slow down inserts, updates and deletes, and extra ones lengthen planning: 7.4 → 10.8 ms. By the measurements keep `rental(client_id)` and `client(last_name text_pattern_ops)`; the index on `started_at`, the ordinary one on `last_name` and the one on `start_station_id` are not needed for this workload.

Say it in Russian: «Индексы занимают место, замедляют вставку и изменение и увеличивают время планирования. По замерам стоит оставить rental(client_id) и индекс с text_pattern_ops.»

How reliable are your measurements?
Насколько надёжны ваши измерения?

Each time is the median of 5 runs after a warm-up, so single outliers do not matter; differences of 5–10 % between steps 0–3 are noise, while the change in step 4 is many times larger. Limits: the cache is hot (only shared hits); with reading from disk the gain from the index would be larger. One computer, no other load. A new psql session adds about 3 ms to the index variants.

What about a composite index (client_id, started_at)?
А что, если сделать составной индекс (client_id, started_at)?

I did not measure it, so I will not claim a result. My expectation: the date condition cuts some of a client's rentals and the sort is by date descending, so a composite index could return the rows in the right order, but with about two rentals per client the gain over the current few milliseconds would be small. It is easy to test with CREATE INDEX and a new EXPLAIN ANALYZE.

4. Work No. 4 – ORM setup: DAO, sessions, transactions

The task (Java reference): a Maven project with Hibernate and the PostgreSQL driver; `hibernate.cfg.xml`; a class `manager.DAO` with a static `ThreadLocal<Session>` and a `SessionFactory`, methods `getSession()`, `begin()` and `commit()`; a class `TestSrvs` extending DAO with `getName(id)`; `Main`; two more methods – a list of values from an ID range, and an insert wrapped in begin/commit using `executeUpdate`. The task sheet does not ask for a report; one was made anyway (7 pages).

4.1 The 30-second pitch

Say this

"The reference stack is Java and Hibernate; I used the agreed alternative, TypeScript with Drizzle ORM. The mapping is one to one: pom.xml is package.json, hibernate.cfg.xml is config.ts, the SessionFactory is a connection pool, the session lives in a static field of the DAO class, and getSession, begin and commit do what the task describes. TestSrvs extends DAO and has three methods: getName by id, getNames for an id range, and addStation, which is wrapped in begin and commit. Main prints the station with id 1, the names of stations 1 to 5, and the number of inserted rows. I checked with a separate psql connection that the inserted row is really committed."

Эталон – Java и Hibernate, я использовал согласованный альтернативный стек: TypeScript и Drizzle ORM. Соответствие прямое: конфигурация – config.ts, фабрика сессий – пул соединений, сессия хранится в статическом поле DAO. TestSrvs наследует DAO и содержит три метода: getName, getNames и addStation, обёрнутый в begin и commit.

4.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
Maven, pom.xml	npm, package.json (dependencies: drizzle-orm, pg)
Hibernate	Drizzle ORM (a query builder with typed table definitions)
JDBC driver for PostgreSQL	pg (node-postgres)
hibernate.cfg.xml	`src/config.ts` (host, port, database, user; defaults point at the project's own server)
entity class with annotations	`pgTable(...)` in `src/schema.ts`
SessionFactory	`Pool` in the static field `DAO.sessionFactory`
ThreadLocal<Session>	static field `DAO.session` (JavaScript runs in one thread)
beginTransaction() / getTransaction().commit()	`BEGIN` / `COMMIT` on the session's connection
createQuery(...).list(); executeUpdate	`session.select()…`; `session.insert(…)` and `rowCount`

4.3 Demo script

1. Start the server (`npm run pg:start`), run `npm start` in pz4. Expected output: 'Площадь Революции', a list of five names, and `1` (one row inserted). Each run adds one station.
2. Open `src\DAO.ts`: point at `sessionFactory` (the pool), `session`, `getSession()` (create on first use), `begin()`, `commit()`, `close()`.
3. Open `src\TestSrvs.ts`: `getName`, `getNames` (BETWEEN, ordered by id) and `addStation` (begin, insert, commit).
4. Prove the commit: in psql run `SELECT id, name FROM station WHERE id > 300;` – the new row is visible to another connection. (Clean up afterwards: `DELETE FROM station WHERE id > 300;`.)

4.4 What you must be able to explain

- What an ORM does and why (object–table mapping, typed queries, less hand-written SQL, parameters sent separately from the SQL).
- SessionFactory versus Session: the factory (here the pool) is created once and gives out sessions (here connections); a session is one unit of work.
- Why begin and commit are needed for the insert: the row becomes permanent and visible to others only after COMMIT.
- How the DAO checks for an open session in `getSession()`, and why `close()` is needed at the end of Main.

4.5 Questions and answers

What is an ORM and why is it used?
Что такое ORM и зачем он нужен?

Object-Relational Mapping connects program objects and database tables, so you write queries through typed code instead of strings. Benefits: fewer mistakes, parameters sent separately (no SQL injection), the same code for different databases. Price: another layer to learn and sometimes less control over the SQL.

Say it in Russian: «ORM связывает объекты программы и таблицы базы данных: запросы строятся типизированным кодом, а не строками. Меньше ошибок и защита от SQL-инъекций.»

Why Drizzle and not Hibernate?
Почему Drizzle, а не Hibernate?

Hibernate is a Java library. The agreed stack for the course work is Node.js and TypeScript, where Drizzle is a typed ORM for PostgreSQL; every Hibernate element of the task has a counterpart (table in the report).

What is the SessionFactory and what is the Session here?
Что такое SessionFactory и Session в вашей реализации?

`DAO.sessionFactory` is a `Pool` of database connections created once from `config.ts`. A session is a Drizzle object built on one connection taken from the pool; `getSession()` creates it on first use and keeps it in `DAO.session`.

What replaces ThreadLocal?
Что заменяет ThreadLocal?

In Java every thread needs its own session, hence ThreadLocal. JavaScript code runs in a single thread, so a static field is enough for this exercise. In work 6, where several requests are served at the same time, the session is kept per request with `AsyncLocalStorage`, which is the real analogue of ThreadLocal.

What do begin() and commit() do? What if you forget commit()?
Что делают begin() и commit()? Что будет, если забыть commit()?

`begin()` sends BEGIN, `commit()` sends COMMIT on the session's connection. Until COMMIT other connections do not see the change, and if the connection is closed without it, the database discards the change. That is why the insert method is wrapped in begin/commit, while the read methods need no explicit transaction.

Is the query built by string concatenation? What about SQL injection?
Запрос собирается конкатенацией строк? Что с SQL-инъекциями?

No. Drizzle sends the SQL text with placeholders and the values separately. I checked it: for `where id = '1; DROP TABLE station'` the generated SQL is `select "name" from "station" where "station"."id" = $1` and the text travels as the parameter, so it is never executed as SQL.

Why does Main call DAO.close()?
Зачем Main вызывает DAO.close()?

The session holds a connection taken from the pool, and the pool holds open sockets; while they exist the Node.js process does not exit. `close()` releases the connection and ends the pool.

What does getNames(from, to) return exactly?
Что именно возвращает getNames(from, to)?

The names of stations whose id is between `from` and `to` inclusive (BETWEEN), ordered by id; for 1 and 5 it is five names: Площадь Революции, Городской сад, Морской вокзал, Приморский бульвар, Заречье.

What is the result of addStation and how can you check it?
Что возвращает addStation и как это проверить?

The number of inserted rows (`rowCount`), that is 1. To check, read the table from another connection, for example in psql: `SELECT id, name FROM station WHERE id > 300`.

5. Work No. 5 – entities, Criteria API, repositories

The task (Java reference): Entity classes for all tables; a DAO with `findAll` through the Criteria API; the work-4 methods rewritten with Criteria and Restrictions; a method with dynamic filters; get, save, delete and a one-to-many demo; Spring Data repositories; a query method and one method with `@Query`. The report must show Entity/DAO/Repository classes, call examples and console output (11 pages).

5.1 The 30-second pitch

Say this

"All nine tables are mapped in model.ts with their columns, keys and foreign keys, and the one-to-many relation district–station is declared in relations.ts on both sides. Criteria is a small class on top of the query builder that collects conditions and ordering, with a Restrictions object for eq, ge, le, between and like. StationDao uses it for findAll, getName, listByIdRange, a prefix search and the dynamic search method: a condition is added only when its parameter is not null. The one-to-many demo creates two stations of a district in a transaction, shows the count 33 → 35 and removes them again. For the Spring Data part there is a generic JpaRepository with findAll, findById, save and deleteById, a StationRepository with a query method and a method with a JOIN, and a DistrictRepository."

Все девять таблиц описаны в model.ts; связь район–станция один ко многим описана с двух сторон в relations.ts. Criteria накапливает условия и сортировку. В методе search условие добавляется только если параметр задан. Репозитории: общий JpaRepository, StationRepository с методом по имени и методом с JOIN, DistrictRepository.

5.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
@Entity, @Table, @Id, @GeneratedValue, @Column	`pgTable('name', { … })` in `src/model.ts` – 9 tables, `generatedByDefaultAsIdentity()`, `notNull()`, `references()`
@OneToMany(mappedBy) / @ManyToOne	`relations()` in `src/relations.ts` with `many` and `one`
session.createCriteria, Restrictions	class `Criteria` and object `Restrictions` in `src/Criteria.ts` (a thin layer over Drizzle's query builder)
AuthorDao-style DAO	`StationDao`, `DistrictDao`
JpaRepository<T, ID>	generic class `JpaRepository`: findAll, findById, save, deleteById
query method findByNameStartingWithIgnoreCase	same method name on `StationRepository`, written by hand with `ilike`
@Query with JPQL join	`findStationsByDistrictName`: an explicit JOIN of station and district

5.3 Demo script

1. `npm start` in pz5 (server running). The output has 15 lines; the key ones: `findAll: 300`; `search("Пл", null, null): 1`; `search(null, 16, 18): 29`; `district Центральный: stations 33 -> 35`, `via relation: 35`, `after delete: 33`; `repository findAll: 300`; `findStationsByDistrictName("Центральный"): 33`.
2. Open `src\model.ts` (first lines: district and station) and `src\relations.ts`.
3. Open `src\Criteria.ts`: `add`, `addOrder`, `list`, `uniqueResult`; then `StationDao.search`: three `if (… !== null)` lines add conditions.
4. Open `src\JpaRepository.ts` and `StationRepository.ts`.

5.4 What you must be able to explain

- How a table is mapped to an entity, and what the foreign key and the relation declaration are for.
- What Criteria and Restrictions are, how a dynamic filter works, and why it is better than concatenating SQL strings.
- What the Repository pattern is and what Spring Data does automatically. **Be honest:** Spring Data derives the query from the method name and generates the implementation; in Node.js there is no such framework, so `findByNameStartingWithIgnoreCase` is written by hand with the same name and the same behaviour.
- The one-to-many demo and fetching: Drizzle loads related rows only when you ask (`with: { stations: true }`); nothing is loaded lazily behind your back.

5.5 Questions and answers

What is an Entity and how is it mapped?
Что такое Entity и как оно отображается?

An entity is a class (in Java) that represents a table row. Here the mapping is a table definition: `pgTable('station', { id, districtId, name, … })` lists the columns, their types and constraints; the types of a row for reading and for inserting are derived from it (`$inferSelect`, `$inferInsert`).

Say it in Russian: «Entity – класс, представляющий строку таблицы. Здесь отображение задаётся определением таблицы: столбцы, типы и ограничения.»

How is the one-to-many relation district–station declared?
Как описана связь «один ко многим» район–станция?

Twice: the table definition of station has a foreign key to district, and `relations.ts` says that a district has `many` stations and a station has `one` district (through `districtId`). Then `getWithStations` returns a district together with its stations.

What is the Criteria API and how is it different from SQL?
Что такое Criteria API и чем оно отличается от SQL?

It builds a query from objects and method calls instead of a text: `criteria.add(Restrictions.eq(station.id, 1))`. The query is checked by the compiler and is built piece by piece, which is ideal for dynamic filters. The same result can be written as SQL, but conditions would have to be glued together as strings.

How does the dynamic filter search() work?
Как работает динамический фильтр search()?

Every parameter may be null. For each non-null parameter one condition is added (`ilike` for the name, `ge` for the lower and `le` for the upper capacity); with no parameters there is no WHERE at all. Measured: name ‘Пл’ → 1 station; capacity 16–18 → 29 stations; name ‘Пл’ and capacity ≥ 10 → 1.

Which Restrictions did you implement?
Какие Restrictions вы реализовали?

`eq`, `ge`, `le`, `between`, `like` and `ilike` – thin names over the Drizzle operators; `Order.asc` and `Order.desc` for sorting.

How did you rewrite the work-4 methods?
Как вы переписали методы четвёртой работы?

`getName(id)` is `Restrictions.eq(station.id, id)` with `uniqueResult()`; `listByIdRange(from, to)` is `Restrictions.between` with ordering by id; the results are the same as in work 4.

What is the Repository pattern and what does JpaRepository give?
Что такое паттерн Repository и что даёт JpaRepository?

A repository hides data access behind a collection-like interface. A generic `JpaRepository` gives any entity findAll, findById, save and deleteById without writing them again; concrete repositories extend it and add their own queries. In Spring Data the framework generates the implementation from an interface; here it is a generic class.

What is a query method? What is @Query?
Что такое query method? Что такое @Query?

A query method is a method whose name is the query: in Spring Data `findByNameStartingWithIgnoreCase(prefix)` becomes ‘name starts with prefix, ignoring case’. `@Query` holds an explicit JPQL or SQL text for what a name cannot express, for example a join. My analogue of the second is `findStationsByDistrictName`, which joins station and district and filters by the district name (33 stations for ‘Центральный’).

How is the one-to-many demo performed?
Как выполнена демонстрация связи один ко многим?

Inside begin/commit two stations are saved for district 1; the count of the district’s stations goes from 33 to 35 (also visible through `getWithStations`); then both are deleted in a second transaction and the count returns to 33.

What is lazy and eager fetching?
Что такое lazy и eager загрузка?

Lazy loads related rows only when they are first used; eager loads them together with the parent. Hibernate has both. Drizzle has no hidden lazy loading: you request the relation explicitly (`with: { stations: true }`), so it is always explicit and eager; this avoids the classic N+1 surprise.

6. Work No. 6 – REST service and transactions

The task (Java reference): a Spring Boot REST service with GET (list), POST (create) and DELETE (by id); JSON; a DTO or projection from several tables; a service layer with `@Transactional` for a composite operation; a rollback demonstration; an optional demo client. The report must show the endpoint URLs, request examples, the transactional service code, a rollback screenshot and the default PostgreSQL isolation level (13 pages).

6.1 The 30-second pitch

Say this

"The service is built with Express on Node.js; the layers are the same as in Spring: routes in app.ts are the controllers, services.ts is the service layer, and Drizzle with the DAO from the earlier works is the data layer. It has GET, POST and DELETE for stations, GET and DELETE and POST for clients, and GET /api/rentals which returns a DTO built from four tables. A missing record gives 404 through an error handler. The transaction is DAO.transactional, my analogue of @Transactional with propagation REQUIRED: begin, run, commit, and rollback on any exception; read methods use a READ ONLY transaction. The composite operation registers a client, a wallet and favourite stations in one transaction; with ?fail=true the service throws after saving, and I showed with psql that none of the three rows exist. Without the error all three are saved. PostgreSQL's default isolation level is READ COMMITTED. The demo client is an HTML page with fetch."

Сервис на Express: маршруты – контроллеры, services.ts – сервисный слой, Drizzle – доступ к данным. GET /api/rentals возвращает DTO из четырёх таблиц. Транзакция – DAO.transactional (аналог @Transactional): фиксация при успехе, откат при исключении. С параметром fail=true после сохранения возникает ошибка, и в базе нет ни клиента, ни кошелька, ни избранных станций. Уровень изоляции по умолчанию – READ COMMITTED.

6.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
Spring Boot, starter-web	Express 5
@SpringBootApplication, SpringApplication.run; 'Started Application on port 8080'	`src/Application.ts`: `app.listen(8080)` and the same message
@RestController, @GetMapping, @PostMapping, @DeleteMapping	`app.get`, `app.post`, `app.delete` in `src/app.ts`
@RequestBody; ResponseEntity.ok / noContent	`express.json()`; `res.json()`, `res.status(204).end()`
DTO record	interface `RentalDto`
@Service, @Transactional, @Transactional(readOnly = true)	classes in `src/services.ts`; `DAO.transactional(fn, { readOnly })`
@ExceptionHandler(EntityNotFoundException) → 404	error-handling middleware: `NotFoundError` → 404
ThreadLocal session per request	`AsyncLocalStorage` in `DAO.run`, started by the `route()` wrapper for every request

6.3 The endpoints

Method	URL	What it does	Answer
GET	/api/stations	list of stations (300)	200
GET	/api/stations/{id}	one station	200 / 404

Method	URL	What it does	Answer
POST	/api/stations	create a station from the JSON body	200
DELETE	/api/stations/{id}	delete a station	204 / 404
GET	/api/rentals?limit=N	DTO from rental + client + station + tariff, newest first	200
GET	/api/clients/{id}	one client	200 / 404
POST	/api/clients[?fail=true]	register a client + wallet + favourite stations in one transaction; `fail=true` throws after saving	200 / 500
DELETE	/api/clients/{id}	delete a client (wallet and favourites go with it)	204 / 404
GET	/	the demo client (public/index.html)	200

POST returns 200 because the task sheet says so (“верните созданный объект с 200 OK”); 201 Created would be the stricter REST choice.

6.4 Demo script

1. Start the database (`npm run pg:start`), then `npm start` in pz6: ‘Started Application on port 8080’. In another terminal or in the browser: `curl http://localhost:8080/api/stations/1`, `curl "http://localhost:8080/api/rentals?limit=2"`, `curl http://localhost:8080/api/stations/999999` (404 with `{"error":"station 999999 not found"}`).
2. POST and DELETE: write the JSON into a file (`station.json`) because Russian text typed straight into a PowerShell or Git Bash command line can arrive as ‘?’: `curl -X POST http://localhost:8080/api/stations -H "Content-Type: application/json" -d @station.json` then `curl -X DELETE http://localhost:8080/api/stations/<id>` (204).
3. The rollback: the same POST to `/api/clients?fail=true` returns 500 `{"error":"test rollback"}`; then in psql count the client, its wallet and favourites (queries in report 6, figures 5 and 6): **0, 0, 0**. Without `fail=true` the answer is 200 and the counts are **1, 1, 2**. Delete the test client afterwards.
4. Or just run `npm run demo`: it starts the service on a free port, runs the whole scenario, checks the database, deletes what it created and prints each step.
5. Open `http://localhost:8080/` in a browser: the demo client shows the stations in a table. Open `src\DAO.ts` (method `transactional`) and `src\services.ts` (`ClientService.register`).

6.5 What you must be able to explain

- REST basics: resources and URLs, the meaning of GET, POST and DELETE, status codes 200, 204, 404, 500, JSON.
- The layers: controller (routes) → service → data access; why business logic and transactions belong in the service.
- A transaction: atomicity (all or nothing); commit and rollback; how `DAO.transactional` works; propagation REQUIRED (a nested call joins the open transaction); read-only transactions.
- The default isolation level of PostgreSQL: READ COMMITTED; what it allows and forbids.
- The demo: what was in the transaction, where the exception is thrown, and what the database contained afterwards.

6.6 Questions and answers

What is REST?
Что такое REST?

An architecture style for web services: resources are addressed by URLs and manipulated with the HTTP methods – GET reads, POST creates, PUT/PATCH changes, DELETE removes – and answered with status codes and usually JSON. The service is stateless: every request carries what is needed.

Say it in Russian: «REST – стиль веб-сервисов: ресурсы адресуются URL, операции – методы HTTP, ответы – коды состояния и JSON. Сервис не хранит состояние между запросами.»

Which status codes do you return and when?
Какие коды ответа вы возвращаете и когда?

200 for a successful GET and POST (as the task sheet says; 201 would be stricter), 204 with an empty body for a successful DELETE, 404 when the record does not exist (from `NotFoundError`), 500 for an unexpected error.

What is a DTO and why do you need it?
Что такое DTO и зачем он нужен?

A Data Transfer Object is a shape made for the answer, not a table row. `RentalDto` combines the rental id, the client's full name, the start station, the tariff code, the start time and the cost from four tables, so the client gets one flat JSON object and the internal table structure stays hidden.

How does the service layer differ from the controller?
Чем сервисный слой отличается от контроллера?

The controller (route) only receives the request and sends the answer; the service holds business logic and transaction boundaries and is independent of HTTP. That way the same service can be called from a route, from the demo script or from a test.

What does @Transactional do, and what is your analogue?
Что делает @Transactional и что у вас ему соответствует?

It wraps a method in a transaction: if the method returns normally the transaction is committed, if it throws a RuntimeException it is rolled back. My analogue is `DAO.transactional(fn, options)`: it runs BEGIN, calls `fn`, runs COMMIT, and on any exception runs ROLLBACK and rethrows. If a transaction is already open it just calls `fn` (propagation REQUIRED).

Say it in Russian: «@Transactional оборачивает метод в транзакцию: при успехе фиксация, при исключении откат. У меня это DAO.transactional: BEGIN, выполнение, COMMIT, а при ошибке ROLLBACK.»

What is propagation REQUIRED and where do you see it?
Что такое propagation REQUIRED и где это видно?

REQUIRED means: join the current transaction if there is one, otherwise start a new one. In `DAO.transactional` the first line checks the scope's `inTransaction` flag and, if it is set, simply returns `fn()`; so a service method called from another transactional method does not open a second transaction.

What does readOnly = true do?
Что делает readOnly = true?

It opens the transaction with `BEGIN READ ONLY`: PostgreSQL then refuses any write in it, which protects read methods from accidental changes and lets the database optimise. All GET methods of my services use it.

Show the rollback. What exactly was rolled back?
Покажите откат. Что именно было отменено?

`ClientService.register` saves a client, its wallet and its favourite stations in one transaction and, with `fail=true`, throws `Error('test rollback')` after saving. `DAO.transactional` catches it, sends ROLLBACK and rethrows; the route's error handler answers 500. Afterwards psql shows 0 clients, 0 wallets, 0 favourites for that phone. Without the flag the answer is 200 and psql shows 1, 1, 2.

What is the default isolation level in PostgreSQL and what does it mean?
Какой уровень изоляции по умолчанию в PostgreSQL и что он означает?

READ COMMITTED (I checked with `SHOW default_transaction_isolation`). Every statement sees only data committed before that statement started. There are no dirty reads, but two identical queries in one transaction may give different results if another transaction commits in between (non-repeatable read). Stricter levels are REPEATABLE READ and SERIALIZABLE.

Say it in Russian: «READ COMMITTED: каждая команда видит только зафиксированные данные. Грязного чтения нет, но возможно неповторяющееся чтение.»

How do you keep one database session per request?
Как у вас обеспечивается одна сессия на запрос?

The `route()` wrapper runs every handler inside `DAO.run`, which creates a scope with `AsyncLocalStorage` (the Node.js analogue of ThreadLocal) and releases the connection at the end. Concurrent requests therefore never share a connection or a transaction.

How is the demo client built?
Как устроен демо-клиент?

`public/index.html` is a static page served by `express.static`: it calls `fetch('/api/stations')`, takes the JSON and fills an HTML table.

What is not in your service? What would you add?
Чего нет в вашем сервисе? Что бы вы добавили?

The code is minimal on purpose, so: **no input validation** – an invalid body or a non-numeric id ends as a database or parsing error and a **500** answer, and I checked that the error text then contains the SQL statement, which a production service should not expose; no authentication; no pagination (GET /api/stations returns all 300 rows); POST returns 200 instead of 201. I would add validation with 400 answers, error messages without SQL, 201 Created and pagination.

Why is POST /api/clients?fail=true a safe way to demonstrate rollback?
Почему параметр fail=true удобен для демонстрации отката?

The error is thrown after all three inserts have run, so it proves that already executed statements are undone; and it is part of the demo only: without the parameter the service behaves normally. In the Java sheet the same is shown by a commented-out `throw new RuntimeException`.

7. If something goes wrong

Problem	What to do
psql says “connection refused”, or `npm start` cannot connect	The server is not running: `npm run pg:start` in any of the folders of works 2–6. The data is the folder `.pgdata`, port 54329.
`pg:start` seems to hang	After a crash or a power-off PostgreSQL recovers first; this can take a minute or more. Wait for “запущен”.
Port 54329 is busy	Another PostgreSQL server of this project is running (the original project’s copy). Stop it first, then start this one.
Russian letters in psql look like ???? or garbage	`$env:PGCLIENTENCODING = 'UTF8'` before psql, and use Windows Terminal. For curl, put Russian JSON into a file and use `-d @file.json`.
`\i` cannot find the file	psql was not started from the exercise folder: `cd` into it first; write paths with forward slashes.
The JOIN demo prints thousands of rows	You ran the whole `04_joins.sql` on the full database. Press Ctrl+C and use the small queries from section 2.2.
The first run is slower than in the report	Cold caches: run the query twice. A new psql session adds a few milliseconds.
The time differs from the report	Normal: the computer, the load and a fresh versus an old session differ. Only the ratio (a few dozen times) and the plan (Seq Scan against Index Scan) are stable.
The plan differs after DROP or CREATE INDEX	Run `ANALYZE rental; ANALYZE client;` and repeat EXPLAIN.
Fewer than five `idx_*` indexes in `\di idx_*`	Run `npm start` in pz3: it recreates the five indexes.
The tables have only about 28,000 records	Someone ran `npm start` in pz2. Run `npm start` in pz3: it loads the data again (about a minute).
The database is damaged or empty	`npm start` in pz2, then `npm start` in pz3: the data is reproducible (fixed seeds), the row counts will match the reports.
Port 8080 is busy (work 6)	Another instance of the service is running. Close it, or find it with `netstat -ano | findstr :8080` and stop that process.
Work 4 keeps adding stations	By design: every `npm start` inserts one. Remove them with `DELETE FROM station WHERE id > 300;` in psql.
The teacher asks to change a surname or a condition	Copy the query from `sql\02_query.sql` into psql, change the value and put `EXPLAIN (ANALYZE, BUFFERS)` in front. Good surnames: `SELECT last_name, count(*) FROM client GROUP BY 1 ORDER BY 2 DESC LIMIT 10;`
Why not the main PostgreSQL on port 5432?	The project has its own cluster in its folder: no password of the main server needed, other databases untouched, fully reproducible.

8. Cheat sheet: numbers and rules

Key numbers

Work	What	Value
1	Tables, DBMS	7 tables (district, station, bicycle, client, tariff, rental, payment); SQLite
1	Rows; CROSS JOIN	10, 12, 14, 12, 10, 16, 12; 38,707,200 rows × 55 columns
2	Constraints in the schema	9 PRIMARY KEY, 12 FOREIGN KEY, 8 UNIQUE, 23 CHECK; 9 tables
2	JOIN rows (base data)	INNER 12; LEFT 13; FULL OUTER 17 (14 pairs + 1 station nobody rode to + 2 unfinished rentals); CROSS 10 × 10 = 100 (the file’s query: 2 × 3 = 6)

Work	What	Value
2	Generated data	60 stations, 600 bicycles, 2,000 clients, 2,000 wallets, 4,770 favourites, 10,000 rentals, 8,591 payments = 28,139 records (10,016 rentals); seed 0.2026
2	ON DELETE	RESTRICT – reference data and history; CASCADE – payment, wallet, favourites; SET NULL – referred_by
3	Data volume	967,801 records: 400,000 rentals, 343,094 payments, 119,387 favourites, 50,000 clients, 50,000 wallets, 5,000 bicycles, 300 stations, 10 districts, 10 tariffs
3	The query	3 tables, 4 conditions, 71 rows; 962 surnames start with ‘Кузн’, 31 with first name ‘Мария’; started_at matches 68.6 % (274,493 rentals)
3	Without indexes	101.7 ms (median); cost 9,692; Parallel Seq Scan on rental; 6,655 pages; planning 7.4 ms
3	With rental(client_id)	4.3 ms in a fresh session (about 1 ms when repeated in one session; 23.4 times faster than 101.7); cost 1,750 (5.5 times lower); 1,025 pages; the index is 3,904 kB and builds in about 0.26 s
3	Indexes without effect here	started_at (8,792 kB), last_name ordinary (376 kB), start_station_id (2,840 kB); text_pattern_ops (376 kB) helped only the client lookup
4	Result of npm start	‘Площадь Революции’; five names (ids 1–5); 1 row inserted (one more station on every run)
5	Results	findAll 300; search 1 / 29 / 1; district Центральный 33 → 35 → 33 stations; 10 districts; findStationsByDistrictName 33
6	Service	port 8080; 9 routes; 200 / 204 / 404 / 500; isolation READ COMMITTED; rollback: 0, 0, 0 rows; success: 1, 1, 2 rows

Key rules

- A foreign key does not create an index automatically in PostgreSQL (a PRIMARY KEY or UNIQUE does). Create an index on a foreign key column if you join or filter by it.
- An index helps when the condition selects a small share of the rows; when it selects 10–30 % or more, the planner chooses a sequential scan.
- A prefix search `LIKE 'abc%'` under a language collation needs an index with `text_pattern_ops` (or a database with the ‘C’ collation).
- Measure before and after; look at the plan (the node type) and the time; differences within 10 % are noise.
- ON DELETE is a rule of the database: RESTRICT for reference data and history, CASCADE for dependent rows, SET NULL for soft links.
- Generated data: parents before children, `setseed` for repeatability, ANALYZE after a bulk load.
- A transaction is all or nothing: COMMIT on success, ROLLBACK on error. Default isolation in PostgreSQL: READ COMMITTED.
- Never glue user input into SQL: an ORM sends values as parameters ($1) separately from the SQL text.
- REST: GET reads (200), POST creates (200 here, 201 is stricter), DELETE removes (204), a missing record is 404.

9. Glossary English – Russian

English	Русский	English	Русский
primary key	первичный ключ	session factory / session	фабрика сессий / сессия
foreign key	внешний ключ	connection pool	пул соединений
referential integrity	ссылочная целостность	entity	сущность

English	Русский	English	Русский
constraint	ограничение	mapping	отображение (маппинг)
one-to-many relation	связь один ко многим	Criteria API	программный интерфейс построения запросов
one-to-one relation	связь один к одному	repository	репозиторий
many-to-many relation	связь многие ко многим	query method	метод запроса по имени
junction table	промежуточная (связующая) таблица	dynamic filter	динамический фильтр
cascading delete	каскадное удаление	lazy / eager loading	ленивая / жадная загрузка
join	соединение (JOIN)	transaction	транзакция
Cartesian product (CROSS JOIN)	декартово произведение	commit / rollback	фиксация / откат
index	индекс	isolation level	уровень изоляции
sequential scan	последовательное чтение	propagation	распространение транзакции
index scan	индексный просмотр	read-only transaction	транзакция только для чтения
query execution plan	план выполнения запроса	REST endpoint	конечная точка REST
plan cost	стоимость плана	DTO (data transfer object)	объект передачи данных
query planner	планировщик	status code	код состояния HTTP
statistics	статистика	request body	тело запроса
selectivity	избирательность условия	service layer	сервисный слой
collation	порядок сортировки	controller	контроллер
ORM (object-relational mapping)	объектно-реляционное отображение	middleware	промежуточный обработчик
DAO (data access object)	объект доступа к данным		